FLOWFORGE

Database Architecture Plan

A forward-looking data model for a Real-Time Order Intelligence Platform

Purpose. Define the target business domains, core entities, relationships, and implementation sequence before writing database models or applying migrations. The architecture should support future capabilities without forcing us to implement every feature at once.

Guiding principle: design for the complete platform, implement in small validated stages, and evolve the schema through reviewed Alembic migrations.

What this plan covers

- Target database domains and table inventory
- Relationships and ownership boundaries
- Implementation roadmap from core schema to real-time intelligence
- Integrity, inventory, money, and event-reliability principles
- A safe path to SQLAlchemy models and Alembic migrations

Status: architecture proposal. The table set and fields should be reviewed against the exact business behaviors before implementation. Optional components are intentionally staged.

1. Goals and scope

FlowForge is planned as a service that tracks orders and their lifecycle, manages stock across warehouses, supports fulfillment and payment workflows, and exposes reliable events for downstream real-time processing and operational analytics.

Questions the platform should eventually answer

• What orders are arriving, and what is each order's current state?

• Which products are selling fastest and where is stock available?

• How much stock is on hand, reserved, available, dispatched, or adjusted?

• Which orders are delayed or blocked in fulfillment?

• What happened to an order or inventory balance, and when?

• Are events late, duplicated, failing, or being retried?

• What are the order volumes, fulfillment times, and inventory trends?

2. Target table inventory

The target architecture below is intentionally broader than the first implementation stage. A table should be implemented when its workflow is in scope, not merely because it appears in this plan.

Domain	Table	Responsibility	Stage
Customer	**customers**	Customer identity and contact details.	1
Customer	**customer_addresses**	Saved addresses; orders preserve address snapshots where required.	1
Catalog	**products**	SKU, product name, description, current catalog price, active status.	1
Catalog	**product_categories**	Category metadata and optional hierarchy for catalog reporting.	1 / optional
Warehouse	**warehouses**	Warehouse identity, code, location, timezone, operational status.	1
Inventory	**inventory**	Current product balance per warehouse: on-hand and reserved quantities.	1
Inventory	**inventory_reservations**	Stock claimed for order items, including quantity and reservation state.	1
Inventory	**inventory_movements**	Auditable stock changes and reasons: receipts, adjustments, dispatches, etc.	1
Orders	**orders**	Order header, customer, status, currency, totals, and timestamps.	1
Orders	**order_items**	Purchased lines, quantity, product references, purchase-time snapshots and totals.	1
Events	**order_events**	Business event history and lifecycle audit trail.	1
Payments	**payments**	Payment attempts/outcomes, amount, currency, provider reference.	2
Fulfillment	**fulfillments**	Warehouse-level work to fulfill an order.	2

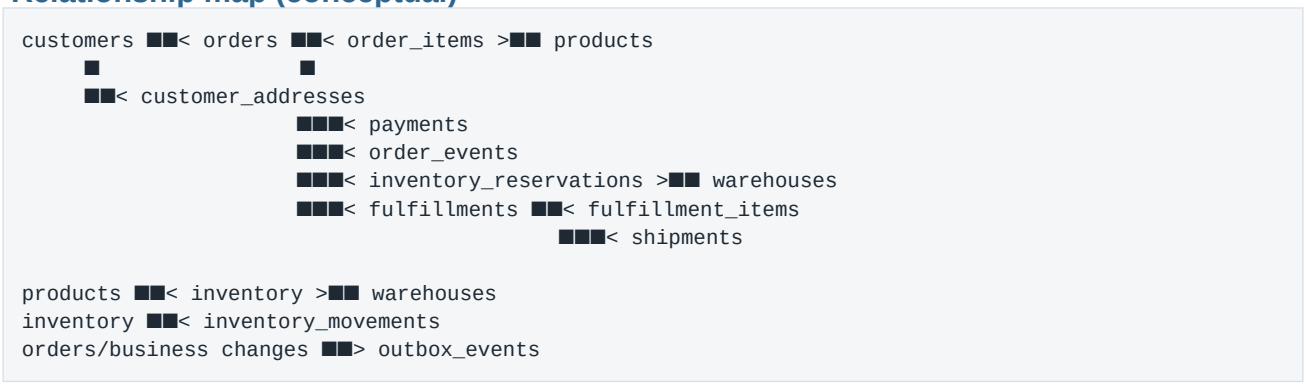
Domain	Table	Responsibility	Stage
Fulfillment	**fulfillment_items**	Quantities of order lines allocated or fulfilled; needed for split/partial fulfillment.	2
Shipping	**shipments**	Carrier, tracking, shipment state, dispatch and delivery timestamps.	2
Event delivery	**outbox_events**	Pending outbound events committed atomically with business changes.	3
API reliability	**idempotency_keys**	Prevents duplicate effects from retried API requests.	3
Returns	**returns**, **return_items**	Return authorization, returned quantities, disposition, and refund workflow.	4 / optional
Operations	**event_processing_attempts**	Processing attempts/failures if not already adequately covered by broker or observability tooling.	3 / conditional
Operations	**inventory_reconciliation_runs**	Records scheduled or external-system stock reconciliation when introduced.	4 / optional

Stage numbers are implementation sequencing, not a statement that every table must be created on day one.

3. Core relationships

Relationship	Design intent
Customer → Orders	One customer can place many orders. An order may retain a customer reference and a snapshot of relevant customer/address details.
Order → Order items	An order contains one or more line items. Each line preserves purchase-time SKU, product name, unit price, quantity, and applicable totals.
Product ↔ Warehouse	Many-to-many relationship represented by inventory balances and stock history; each balance is unique for a product/warehouse pair.
Order item → Reservation	Reservations associate a requested order line with a supplying warehouse and reserved quantity.
Inventory → Movements	Balances provide current state; movements explain how the state changed. Balance updates and ledger writes must be transactionally consistent.
Order → Payments	An order can have multiple payment attempts or transactions; payment state is not identical to order state.
Order → Fulfillments → Shipments	An order can be fulfilled in parts or from multiple warehouses. Fulfillment items record exact quantities when partial/split fulfillment is supported.
Business change → Outbox event	The outbox records outbound messages in the same transaction as the business change, reducing the chance of lost published events.

Relationship map (conceptual)



The diagram is conceptual. Exact foreign keys and cardinalities will be specified in the model design and migration.

4. Architectural rules to establish early

Stable identifiers. Use generated primary keys for internal relationships and unique business identifiers (for example, order_number) for external reference.

Historical snapshots. Catalog prices and product descriptions can change. Store the purchase-time values needed to preserve the meaning of past orders. Preserve address snapshots where later edits must not alter an order.

Money. Use fixed-precision numeric/decimal types, explicit currency, and defined rounding rules. Avoid binary floating-point for monetary values.

Inventory semantics. Define on-hand, reserved, and available quantities explicitly. Available stock is normally on_hand minus reserved, subject to the agreed inventory model.

Concurrency safety. Reserve stock using database transactions and row locks or conditional updates. Do not rely on an application-only read-then-write check to prevent overselling.

Auditability. Keep inventory movement and business event histories separate from current state. Define which records are append-only and how corrections are represented.

Event reliability. Introduce an outbox when publishing database changes to a broker. Consumers should tolerate duplicate delivery through idempotent processing.

Integrity. Use foreign keys, unique constraints, check constraints, and indexes to encode invariants in PostgreSQL, not only in application code.

Schema evolution. Version schema changes with Alembic; review generated migrations and test upgrade paths before applying them.

Data ownership. Decide which service owns each write. Analytics should derive from authoritative operational data rather than becoming a second conflicting source of truth.

5. Implementation roadmap

Stage 1 — Core data foundation

• Implement customers, customer_addresses, products, product_categories (if category metadata is required), warehouses, orders, order_items, inventory, inventory_reservations, inventory_movements, and order_events.

• Define types, relationships, constraints, indexes, timestamps, and delete behavior.

• Generate and review the first Alembic migration; apply it only to the FlowForge development database.

• Verify table definitions, foreign keys, constraints, and a small set of model tests.

Stage 2 — Order lifecycle

• Implement order creation and retrieval, validated status transitions, purchase snapshots, request validation, and idempotency.

• Make order creation and stock reservation succeed or fail together in a transaction.

• Test concurrent reservations, cancellation/release behavior, and inventory consistency.

Stage 3 — Payments and fulfillment

• Add payments, fulfillments, fulfillment_items, and shipments.

• Support payment attempts, warehouse allocation, partial fulfillment, and shipment tracking.

• Introduce explicit workflows for failures and cancellation.

Stage 4 — Real-time event processing

• Add outbox_events and integrate a message broker and publisher.

• Implement consumer idempotency, retries, dead-letter handling, and event schema/versioning practices.

• Decide which processing metadata belongs in PostgreSQL versus the broker/observability system.

Stage 5 — Order intelligence

• Build metrics for order volume, status transitions, inventory availability, fulfillment latency, and processing failures.

• Start with SQL queries or views; add materialized views or a dedicated analytical store only when scale and query patterns justify them.

• Define freshness and correctness expectations for each metric.

Stage 6 — Production readiness

• Strengthen authentication/authorization, secrets management, structured logging, metrics, tracing, and alerting.

• Test backup/restore, migration rollback strategy, capacity, failure recovery, and deployment practices.

• Add returns, reconciliation, and other optional domains as requirements are established.

6. Validation checklist before the first migration

■ All Stage 1 entities and relationships have been reviewed.

■ Required/nullable columns and default values are intentional.

■ Unique constraints exist for business identifiers and product/warehouse inventory balances.

■ Foreign-key delete behavior is explicit; historical order and event data cannot be accidentally cascaded away.

■ Check constraints cover positive quantities and valid numeric ranges where appropriate.

■ Indexes support known access patterns without indexing every column.

■ Order totals, tax/discount handling, currency, and rounding rules are defined.

■ Inventory reservation and movement semantics are consistent and testable.

■ Alembic metadata includes every intended model and autogenerate output has been reviewed.

■ The migration has been tested against the FlowForge development database only.

7. Immediate next steps

• Write a detailed Stage 1 schema specification with columns, PostgreSQL types, primary/foreign keys, constraints, indexes, and delete behavior.

• Implement SQLAlchemy models from the reviewed specification.

• Ensure Alembic imports the model metadata and uses the intended FlowForge database URL.

• Generate and inspect the initial migration; do not apply it until it is reviewed.

• Apply and verify the migration in the FlowForge development environment, then add tests.

Scope guardrail: this plan is a target architecture, not a mandate to build every subsystem immediately. Add each table when its workflow is being implemented, while preserving the relationships and invariants defined here.